Lab 09 — Code Review

Menemukan Risiko Sebelum Production

Checkout naive vs improved:
stok, retry, partial state, dan batas bukti.



Lukman / lukman-ss

Checkout success belum membuktikan correctness

Response sukses adalah hasil API. Review harus memeriksa state yang ditinggalkan oleh operasi bisnis.

STOK

Pengurangan stok dapat tersimpan meskipun order creation gagal.

ORDER

Retry tanpa perlindungan dapat menjalankan checkout lagi.

AKSES

Caller harus berwenang terhadap cart yang diminta, bukan hanya membawa user ID.

Baca requirement dan failure path sebelum mempercayai success.

Requirement sebelum membaca diff

Nilai perubahan dari behavior yang diminta. Banyak baris berubah tidak otomatis berarti banyak risiko.

01 / REQUIREMENT

Stock >= 0. Order dan reservasi harus berhasil bersama. Retry menjaga hasil. Cart hanya milik principal.

02 / BEHAVIOR

Apa yang berubah pada jalur sukses, error, retry, dan akses lintas user?

03 / REVIEW

Requirement → behavior → failure mode → impact → fix → test.

Mulai dari aturan bisnis, lalu cari cabang kode yang melindunginya.

Alur naive meninggalkan celah

Stock update, order creation, dan notification berjalan tanpa satu boundary bisnis.

PRODUCT LOOKUP

GetProduct di loop; error lookup membuat item dilewati, tetapi item tetap masuk order melalui toOrderItems.

STATE BERUBAH

UpdateStock error diabaikan. Stok dapat berubah sebelum order dibuat.

SUCCESS PALSU

CreateOrder dan notification error diabaikan. GetCart error tetap diteruskan oleh kode Go aktual.

```
_ = c.repo.CreateOrder(ctx, order)
_ = c.notify.SendOrderConfirmation(
    ctx, userID, orderID)
```

Tidak semua error diabaikan; verifikasi cabang aktual, bukan komentar.

Invariant: stock >= 0

Aturan ini harus tetap benar setelah checkout sukses, gagal, atau saling overlap.

NAIVE

Pengurangan delta diterima tanpa kondisi stock cukup. Quantity berlebih dapat membuat stok negatif bahkan tanpa konkurensi.

IMPROVED

Quantity harus positif. ReserveStock dalam transaksi memeriksa sisa stok setelah reservasi yang terakumulasi.

BUKTI

Test stok tidak cukup memeriksa stok tetap 5 dan order tetap 0 ketika quantity yang diminta 10.

Uji state akhir, bukan hanya apakah function mengembalikan error.

Dua pembeli, satu unit

Test naive mengatur barrier agar kedua product read selesai sebelum checkout melanjutkan.

KONDISI AWAL

Stock p1 = 1. User 1 dan user 2 masing-masing meminta quantity 1.

↓

OPERASI NAIVE

Kedua request menjalankan delta -1 tanpa kondisi stock cukup.

↓

ASSERTION TEST

Dua logical success dan stok negatif. Ini ekspektasi test yang dibaca, bukan hasil run yang diklaim.

Overselling berarti behavior bisnis salah walaupun request terlihat berhasil.

Data race berbeda dari business race

Sinkronisasi memory dan perlindungan invariant adalah dua bukti yang berbeda.

DATA RACE

Concurrent memory access tanpa sinkronisasi yang benar. Race detector membantu memeriksa kategori ini.

BUSINESS RACE

Urutan operasi merusak aturan bisnis. Mock UpdateStock memakai mutex, tetapi tetap dapat menghasilkan stock negatif.

VERIFIKASI

go test -race untuk memory safety; assertion jumlah order, sukses, dan stok untuk invariant checkout.

Race detector tidak membuktikan bebas overselling.

Transaction boundary menjaga satu hasil

Reservasi semua item dan pembuatan order berada dalam callback yang sama.

BERHASIL

Callback selesai, lalu mock commit menerapkan stok dan order.

GAGAL

ReserveStock atau CreateOrder error menghentikan callback. Perubahan staged tidak menjadi hasil bisnis.

```
// Dalam callback; kode sekitar dipersingkat
if createErr := tx.CreateOrder(ctx, order);
    createErr != nil {
    return createErr
}
```

BEGIN/COMMIT saja tidak menetapkan aturan concurrency stok.

Order gagal: stok harus tetap utuh

Failure injection memeriksa apakah checkout meninggalkan partial state.

SETUP

Dua product memiliki stok 10 dan 1. Order repository diminta gagal saat CreateOrder.

ASSERTION

Checkout error. Stok p1 tetap 10, p2 tetap 1, dan tidak ada order yang dibuat.

RETRY

Test terpisah memeriksa tidak ada notification pada kegagalan, kemudian retry dapat membuat satu order.

Rollback dinilai dari state, bukan hanya keberadaan callback transaksi.

Reservasi: check dan mutation bersama

Mock transaction mengakumulasi deduction per product. README memberi ilustrasi operasi kondisional untuk database.

KONDISI

Stok harus cukup sebelum pengurangan diterapkan. Product yang muncul berulang diperiksa secara kumulatif.

SQL DOKUMENTASI

Periksa affected rows; 0 berarti kondisi reservasi tidak terpenuhi. SQL berikut bukan adapter yang diuji dalam lab.

```
UPDATE products
SET stock = stock - $1
WHERE id = $2
  AND stock >= $1;
```

Gunakan mekanisme yang menjaga invariant pada boundary data sebenarnya.

Mock menserialkan transaksi

Satu mutex pada MockTransactionManager ditahan selama callback dan commit.

CAKUPAN

Request yang memakai manager sama menjalankan transaksi bergantian. Ini boundary global demonstrasi pada manager tersebut.

BATAS

Tidak ada adapter database production. Test tidak mengukur isolation level, MVCC, atau lock contention database.

DETAIL TEST

DeterministicNoRace memasang read hook, tetapi GetProducts pada improved tidak memanggil hook itu.

Assertion invariant berguna; jangan memperluasnya menjadi bukti database production.

Retry dapat membuat order kedua

Timeout client tidak memberi tahu apakah operasi bisnis sudah selesai di server.

NAIVE

Tidak ada idempotency claim. Pemanggilan ulang dapat mengurangi stok dan membuat order lagi.

IMPROVED

Key di-scope per user. Claim atomik menjaga kepemilikan proses; completed result dapat direplay.

ASSERTION

DuplicateRequestReturnsSameResult memeriksa order ID yang sama dan hanya satu order setelah dua request.

Bedakan identitas operasi dengan jumlah percobaan request.

PROCESSING bukan replay

Claim dan hasil tersimpan mempunyai kontrak yang berbeda.

KEY BARU

Claim membuat record PROCESSING dengan hash. Caller pertama melanjutkan checkout.

↓

KEY SEDANG DIPROSES

Hash sama → ErrDuplicateRequest. Hash berbeda → ErrIdempotencyConflict.

↓

KEY SELESAI

COMPLETED dengan response dapat direplay. MarkCompleted berlangsung setelah business commit.

Concurrent same-key test memeriksa satu efek bisnis; kedua caller tidak wajib sukses.

Hash command, bukan isi cart

CheckoutCommand hanya berisi CartOwnerID dan IdempotencyKey. Cart state tidak masuk hash.

URUTAN AKTUAL

Get COMPLETED → replay awal. Jika belum selesai: load cart → validasi → hash → Claim.

CONFLICT

Repository Claim membandingkan hash. Jalur replay awal service tidak membandingkan RequestHash.

BATAS

Command sekarang dan ownership check membatasi variasi payload. Perlu review ulang jika command diperluas; perubahan cart tidak dideteksi hash.

README menyederhanakan urutan; claim aktual terjadi setelah cart loading.

Ownership dan key scoping

Authorization melindungi cart. Per-user scoping memisahkan operasi dua user.

AUTHORIZATION

Principal.UserID harus sama dengan CartOwnerID. Pemeriksaan terjadi sebelum cart loading dan replay.

SCOPING

checkout:{userID}:{key} membuat string key yang sama pada dua user menjadi dua operasi.

BATAS IDENTITAS

Middleware authentication tidak tersedia di lab. Equality check tidak memvalidasi principal ID kosong.

```
if principal.UserID != cmd.CartOwnerID {
    // log dipersingkat
    return nil, ErrForbidden
}
```

Test akses terlarang memeriksa ErrForbidden, stok utuh, dan nol order.

N+1 ada pada product lookup

Naive melakukan single-product lookup sebanyak item yang dibaca.

LOOP

GetProduct dipanggil per item cart; product berulang juga dilookup berulang.

TERMINOLOGI

Banyak UPDATE adalah pola write terpisah. Itu tidak otomatis disebut N+1 relation lookup.

ASSERTION

Tiga item pada test naive → tiga GetProduct call, nol GetProducts call.

```
product, err := c.products.GetProduct(
    ctx, item.ProductID)
```

Sebut operasi yang berulang agar finding performance tetap konkret.

Batch loading punya bukti terbatas

Improved mengumpulkan unique product ID lalu menjalankan satu GetProducts.

CALL COUNTER

BatchLoadsProducts memeriksa satu batch call dan nol single call. Test unique IDs memeriksa dua ID pada fixture berulang.

HARGA

Subtotal memakai product.UnitPrice, bukan cart.UnitPrice. Mock product memiliki harga tetap 1000.

VALIDASI BELUM LENGKAP

Quantity positif diperiksa. Harga negatif dan overflow perkalian / total belum memiliki guard.

Call count bukan benchmark database; price source bukan validasi uang lengkap.

Error perlu mempertahankan sebab

Caller dan investigator perlu membedakan domain failure dari infrastructure failure.

PROPAGATION

Load cart dan batch error memakai wrapping. Test infra error hanya memeriksa agar tidak dilabeli ErrProductNotFound.

RELEASE

Transaction error digabung dengan Release error melalui errors.Join. Missing-product branch hanya mencatat Release error.

LOGGING

Call membawa userID, key, orderID, dan error sesuai cabang. MockLogger menyimpan pesan saja, bukan field terstruktur.

Jangan mengklaim semua cleanup error atau field log telah dibuktikan test.

Notification setelah commit

Pengiriman synchronous baru dicapai setelah business commit dan MarkCompleted berhasil.

BEST EFFORT

Send error dicatat; order yang committed tidak dibatalkan dan checkout tetap mengembalikan success.

FAILURE WINDOW

Crash sebelum delivery dapat menghilangkan pesan. Completed replay tidak mengirim notification lagi.

BATAS LAB

Mock sender selalu sukses. Failure pengiriman dan durable delivery tidak dibuktikan. Outbox belum diimplementasikan.

Post-commit mengubah boundary; ia tidak membuat delivery durable.

Commit ≠ finalisasi idempotency

Order dapat tersimpan sementara record idempotency belum COMPLETED.

01 / COMMIT

Stok berkurang dan order tersimpan.

↓

02 / MARK GAGAL

ErrIdempotencyFinalize dikembalikan. Record mock masih PROCESSING; notification tidak dicapai.

↓

03 / ASSERTION

Test mengharapkan stock 8 dari 10, satu order, nol notification, dan record tidak COMPLETED.

Error setelah commit bukan bukti rollback. Recovery belum diimplementasikan.

Severity mengikuti risiko

README menggunakan impact × likelihood × blast radius sebagai kerangka prioritas.

BLOCKER / CORRECTNESS

Stok negatif atau akses lintas user dapat merusak invariant dan boundary keamanan.

MAJOR / FAILURE PATH

Partial state, duplicate operation, dan error yang tertelan mengubah asumsi caller.

MINOR / SUGGESTION

Lookup berulang dinilai sesuai workload. Naming, formatting, dan abstraction tidak mendapat bobot yang sama.

Jumlah baris kode dan jumlah komentar bukan ukuran severity.

Test melindungi behavior yang berubah

Go tidak tersedia di lingkungan pembuatan konten ini. Test dibaca; tidak ada klaim run atau benchmark.

ASSERTION TERSEDIA

Stok cukup / kurang, rollback multi-product, concurrent stock reservation, same-key retry, ownership, dan batch calls.

BATAS COVERAGE

Notification mock selalu sukses; logger tidak menyimpan fields. Tidak ada test database production atau guard price/overflow.

BATAS IMPROVED

ID counter milik satu service instance. Tidak ada TTL/recovery PROCESSING, DB unique constraint, atau exactly-once umum.

Nyatakan apa yang diuji, apa yang disimulasikan, dan apa yang belum dibuktikan.

Review adalah perlindungan behavior

Temuan harus menghubungkan baris kode dengan risiko dan bukti yang dapat diperiksa.

MENTAL MODEL

Code review reduces risk by protecting behavior and business invariants.

URUTAN KERJA

Requirement → behavior → failure mode → impact → simplest sufficient fix → test evidence.

PELAJARAN

Transaksi, claim, dan batch loading harus dinilai dari semantics implementasinya. Improved tetap boleh memiliki finding baru.

Verifikasi kode aktual; daftar checklist tidak menggantikan analisis.

Baca kode. Periksa risiko. Lindungi invariant.

Software Engineering Lab
Lab 09 — Code Review

README.md • checkout_naive.go
checkout_improved.go • domain.go
mock.go • checkout_test.go • go.mod

REPOSITORY

github.com/lukman-ss/software-engineering-lab
/tree/main/labs/09-code-review

Lukman / lukman-ss

Sumber diperiksa pada commit
82af988b89439fa92b1200406c5afbd59ce72a56
Test dibaca, tidak dijalankan.